

System-kolejkowania-pacjentow-z-priorytetami



Przedmiot	RAIM – Rozwój aplikacji internetowych w medycynie (2025/2026)
Temat	Temat 2 – System kolejowania pacjentów z priorytetami
Etap	Etap 4 – Finalna obrona
Rok studiów	3
Prowadzący	dr inż. Anna Jezierska
Autorzy	Adam Sikorsi, Mateusz Grochowalski
Uczelnia	Politechnika Gdańska, Wydział ETI
Katedra	Katedra Inżynierii Biomedycznej (KIB)

Spis treści

1. [Analiza potrzeb i wymagań klinicznych](#)
2. [Instrukcja lokalnego uruchomienia i testu](#)
3. [Projekt architektury systemu](#)
4. [Symulacja zaburzeń](#)
5. [Instrumentacja](#)
6. [Omówienie zagadnień współbieżności \(race condition\)](#)
7. [Analiza Fairness i Mechanizm Aging](#)
8. [Analiza Kompromisów Implementacyjnych](#)
9. [Które Kompromisy Są Dopuszczalne?](#)
10. [Wnioski](#)

1. Analiza potrzeb i wymagań klinicznych

1.1 Identyfikacja problemu

W środowisku szpitalnym, a szczególnie w izbach przyjęć (SOR), poradniach i oddziałach intensywnej terapii - zarządzanie kolejką pacjentów jest istotnym problemem. Tradycyjne podejście FIFO (first-in, first-out) bez uwzględnienia stanu klinicznego pacjenta może prowadzić do poważnych zagrożeń zdrowotnych: pacjent w stanie zagrożenia życia może oczekiwać za pacjentem z mniej groźną dolegliwością.

Niniejszy projekt symuluje system kolejowania pacjentów oparty na priorytetach klinicznych, który w kolejnych etapach został rozszerzony o race condition. Dodatkowo zostanie przeprowadzona analiza fairness i

mechanizmu aging w kontekście naszego systemu.

Etap 1 implementuje podstawowy wariant kolejki **FIFO** za pomocą losowej generacji pacjentów (według rozkładu Poissona) i obsługi przez operatora.

Etap 2 Dynamiczne zmiany **priorytetów** przez wielu operatorów.

Etap 3 Wymuszenie race condition i implementacja mechanizmu blokady lub wersjonowania. **Etap 3** Analiza fairness i mechanizm aging.

1.2 Określenie użytkowników systemu

Pacjent - Osoba dodawana do kolejki, dodanie do kolejki, podgląd realizacji przyjęcia.

Operator medyczny - Pielęgniarka/lekarz przyjmujący, przyjęcie następnego pacjenta, aktualizacja statusu.

1.3 Analiza ryzyk

#	Ryzyko	Prawdopodobieństwo	Wpływ	Redukcja
R1	Race condition przy jednoczesnym pobieraniu pacjenta przez wielu operatorów oraz zmian priorytetu	Wysokie	Krytyczny	Mechanizm blokad
R2	Starvation pacjentów z niskim priorytetem (ryzyko nieprzyjęcia)	Wysokie	Wysoki	Mechanizm aging
R3	Utrata danych kolejki przy restarcie systemu	Wysokie	Wysoki	Zapis danych w SQLite
R4	Błędy przy aktualizacji priorytetów	Średnie	Wysoki	Wersjonowanie rekordów
R5	Przeciążenie systemu przy dużym napływie pacjentów	Niskie	Średni	Logi i monitoring

2. Instrukcja lokalnego uruchomienia i testu

Kroki uruchomienia lokalnego:

1. Zainstaluj requirements.txt:
 - `pip install -r requirements.txt`
2. Uruchom aplikację:
 - `python run.py` Aplikacja nasłuchuje domyślnie na `http://0.0.0.0:5000` (lokalnie: `http://127.0.0.1:5000`).
3. Wejdź w przeglądarce na `http://127.0.0.1:5000` aby zobaczyć UI.

Uruchamianie testu race-condition (skrypt test.py):

1. Upewnij się, że serwer działa.
2. W folderze projektu uruchom:
 - `python scripts/test.py`
3. W razie potrzeby zmodyfikuj w `scripts/test.py`:

- TEST_MODE, THREADS_PER_ROUND, TEST_ROUNDS — parametry testu.

4. Interpretacja wyników:

- 200 — sukces
- 409 — conflict (race condition)
- 429 — cooldown (intencjonalna ochrona, nie race)
- -1 — błąd połączenia / timeout

Uwagi:

- W celach demonstracyjnych w projekcie istnieje przełącznik ochrony race condition w prawym górnym rogu interfejsu.

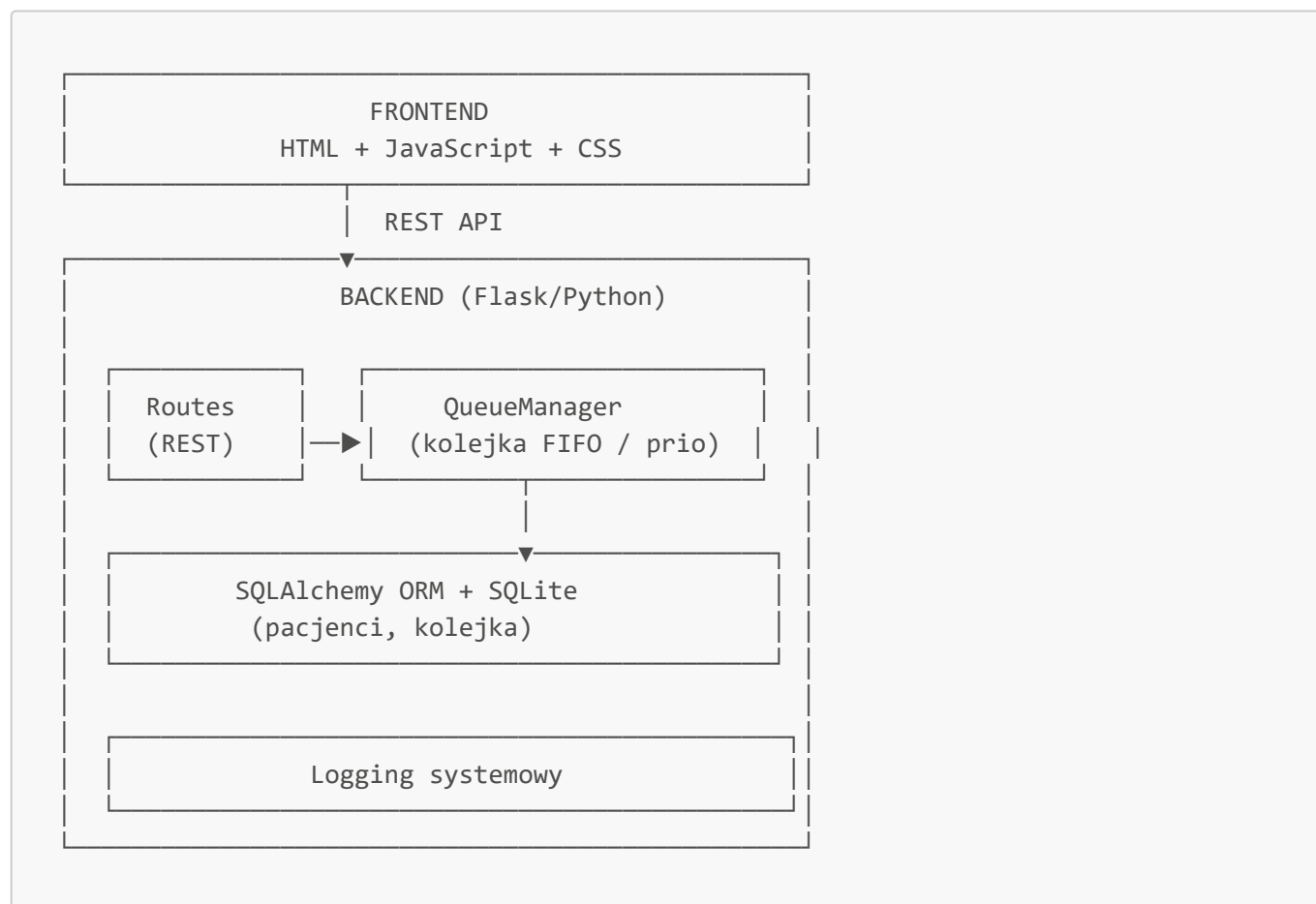
3. Projekt architektury systemu

Uruchamianie lokalnie przez <http://127.0.0.1:5000>

Uruchamianie zdalnie przez <https://system-kolejkowania-pacjentow-z-xuty.onrender.com>

3.1 Przegląd architektury

System zbudowany jest w architekturze **klient-serwer** z komunikacją REST API:



3.2 Model danych (Etap 1)

```
Pacjent
├── id (int)
├── imię (string)
├── nazwisko (string)
├── priorytet
├── czas przybycia (datetime)
└── status
```

```
QueueEntry
├── id (int)
├── priorytet
├── czas przybycia (datetime)
├── czas w kolejce (datetime)
└── czas przyjęcia (datetime)
```

4. Symulacja zaburzeń

Podczas realizacji projektu napotkano dwa istotne problemy charakterystyczne dla systemów kolejkowych i współbieżnych: starvation oraz race condition.

4.1 Starvation

Starvation oznacza sytuację, w której element o niższym priorytecie może być przez długi czas pomijany, ponieważ system stale preferuje elementy o priorytecie wyższym. W kontekście naszego projektu oznacza to, że pacjent o niskim priorytecie mógł oczekiwać znacznie dłużej, jeśli w kolejce nieustannie pojawiali się pacjenci pilniejsi. Zjawisko to jest dobrze znane z teorii planowania zadań w systemach operacyjnych i stanowi naturalne ryzyko w algorytmach priorytetowych.

4.2 Race Condition

Drugim zaobserwowanym problemem był race condition, czyli błąd współbieżności pojawiający się wtedy, gdy dwie osoby jednocześnie modyfikują te same dane. W naszym przypadku dotyczyło to równoczesnej zmiany priorytetu tego samego pacjenta przez dwóch użytkowników systemu. Przykładowo, jeśli pacjent miał ustawiony priorytet 4, jedna osoba mogła kliknąć obniżenie priorytetu, a druga jego podwyższenie. Ze względu na opóźnienie aktualizacji danych w bazie oraz brak pełnej synchronizacji widoków, system mógł chwilowo pokazać priorytet 3, a następnie przeskoczyć na 5. Tego typu zachowanie jest przykładem konfliktu współbieżnych zapisów i pokazuje, że końcowy stan danych zależał od kolejności wykonania operacji.

Opisane problemy pokazują, że projektowanie systemów kolejkowych nie ogranicza się jedynie do ustalenia zasad obsługi pacjentów, ale wymaga również uwzględnienia sprawiedliwości działania algorytmu oraz odporności na jednoczesny dostęp wielu użytkowników. W praktyce ograniczanie starvation może wymagać zastosowania mechanizmów zwiększania priorytetu wraz z czasem oczekiwania, natomiast redukcja race condition wymaga lepszej kontroli współbieżności, na przykład blokad, transakcji lub mechanizmów wykrywania konfliktów zapisu.

5. Instrumentacja

W analizie wydajności systemu instrumentacja oznacza zastosowanie dodatkowych mechanizmów rejestrujących zdarzenia oraz czasy ich występowania w trakcie działania aplikacji. Jej celem jest zebranie danych, które pozwalają mierzyć i oceniać zachowanie systemu pod względem wydajnościowym, na przykład czas obsługi operacji, czas oczekiwania lub opóźnienia komunikacyjne.

5.1 API latencja last

W odniesieniu do latencji instrumentacja umożliwia wyznaczenie czasu potrzebnego na realizację konkretnego działania, na przykład zapisania zmiany priorytetu pacjenta do bazy danych albo odświeżenia widoku kolejki po stronie użytkownika. Porównanie momentu rozpoczęcia i zakończenia operacji pozwala obliczyć opóźnienie, a analiza wielu takich pomiarów umożliwia ocenę wydajności całego systemu lub jego poszczególnych komponentów.

W naszym programie tuż po wejściu do endpointu zapisywany jest czas startu. Następnie wykonywana jest logika endpointu - między t_0 a końcem pomiaru działają operacje backendowe, m.in.:

-próba przyjęcia pacjenta z kolejki, -usunięcie pacjenta z patients.db, -aktualizacje stanu.

Koniec pomiaru i obliczenie latencji.

5.2 API latencja avg

Średni czas obsługi żądania API (w ms), liczony z ostatnich próbek.

5.3 API jitter

Istotnym uzupełnieniem pomiaru latencji jest pomiar jitteru, czyli zmienności opóźnienia pomiędzy kolejnymi wykonaniami tej samej lub podobnej operacji. Jitter pokazuje, na ile stabilny czasowo jest system: nawet jeśli średnia latencja pozostaje akceptowalna, duże wahania pomiędzy kolejnymi pomiarami mogą świadczyć o niestabilności działania, przeciążeniu lub problemach ze współbieżnością. Dzięki instrumentacji możliwe jest więc nie tylko określenie średniego czasu odpowiedzi, ale także ocena, czy system działa w sposób przewidywalny i powtarzalny.

6. Omówienie zagadnień współbieżności (race condition)

6.1 Współbieżność

Współbieżność oznacza wykonywanie wielu operacji „w tym samym czasie” (np. przez wiele wątków lub wielu użytkowników systemu).

W aplikacjach webowych oznacza to, że serwer może obsługiwać kilka żądań jednocześnie, które odwołują się do tych samych danych.

6.2 Czym jest race condition

Race condition (wyścig) występuje wtedy, gdy wynik końcowy zależy od kolejności wykonania równoległych operacji na wspólnym zasobie.

Jeżeli nie ma poprawnej synchronizacji, dwa żądania mogą odczytać ten sam stan „przed zmianą” i zapisać kolidujące wyniki.

6.3 Jak race condition wygląda u nas projekcie

W projekcie występują dwa główne miejsca podatne na wyścig:

1. Przyjęcie pacjenta

Dwóch operatorów może równocześnie próbować przyjąć tego samego pierwszego pacjenta z kolejki.

2. Zmiana priorytetu

Dwóch operatorów może jednocześnie zmieniać priorytet tego samego pacjenta (np. jeden zwiększa, drugi zmniejsza).

6.4 Wymuszenie race condition

Do celów dydaktycznych race condition jest świadomie wzmacniany przez:

- równoległe żądania (test wielowątkowy),
- celowe opóźnienie (`time.sleep(0.5)`) w sekcjach krytycznych przy wyłączonej ochronie. Celowo poszerza okno krytyczne między odczytem a zapisem, więc drugi request ma większą szansę „wejść w środek” operacji.

Dzięki temu łatwo odtworzyć konflikt i obserwować jego skutki w API i metrykach.

6.5 Mechanizm blokady / wersjonowania

W projekcie zastosowano dwa podejścia:

- **Blokada (lock)**

Przy włączonej ochronie operacje wykonywane są w sekcji krytycznej, co serializuje dostęp (operacje nie idą jednocześnie, tylko jedna po drugiej) do kolejki i ogranicza konflikty.

- **Cooldown po zmianie priorytetu (2 sekundy, przy włączonej ochronie)**

Po udanej zmianie priorytetu system chwilowo blokuje kolejną zmianę tego samego rekordu.

To celowy mechanizm ograniczający „przepychanie” jednego pacjenta przez wielu operatorów w tej samej chwili.

Odpowiedź z API dla takiej blokady to **429** (cooldown/rate-limit), co **nie jest race condition**.

- **Wykrywanie konfliktu zapisu (wariant wersjonowania logicznego, tryb bez ochrony)**

Ten mechanizm jest używany przede wszystkim w trybie **bez blokady** (gdy celowo dopuszczamy współbieżny zapis dla demonstracji race condition).

Dla zmiany priorytetu wykorzystywany jest znacznik ostatniego autora (`_last_writer`) oraz porównanie stanu przed/po (`old_priority` vs bieżący `priority`).

Jeśli w międzyczasie inny operator zmieni rekord, backend zwraca **"conflict"**, a endpoint mapuje to na HTTP **409**.

6.6 Porównanie przed i po poprawce (test)

Do porównania używany jest skrypt `test.py`, który:

- loguje dwóch użytkowników,
- uruchamia równoczesne żądania w rundach (**admit** albo **priority**),
- zbiera statusy odpowiedzi (**200**, **409**, **429**) i podsumowanie.
200 - żądanie zostało obsłużone poprawnie przez API.
409 - Conflict (konflikt współbieżności, race condition).
429 - cooldown/rate-limit po zmianie priorytetu (mechanizm ochronny, **nie** race condition).

Interpretacja wyników:

- **Przed poprawką / przy wyłączonej ochronie:** częstsze konflikty i niestabilność wyniku (więcej sytuacji wyścigu).

Przyjęcie pacjenta:

Wyniki: tryb=admit, wątki=2

Łączna liczba rund: 10

Wszyscy 200: 0 ← wyścig niezauważony

Przynajmniej jeden 409: 10 ← wykryty wyścig

Zmiana priorytetu:

Wyniki: tryb=priority, wątki=2

Łączna liczba rund: 10

Wszyscy 200: 1 ← wyścig niezauważony

Przynajmniej jeden 409: 9 ← wykryty wyścig

- **Po poprawce / przy włączonej ochronie:** stabilniejszy, deterministyczny przebieg operacji (mniej konfliktów logicznych).

Przyjęcie pacjenta:

Wyniki: tryb=admit, wątki=2

Łączna liczba rund: 10

Wszyscy 200: 10 ← wyścig niezauważony

Przynajmniej jeden 409: 0 ← wykryty wyścig

Zmiana priorytetu:

Wyniki: tryb=priority, wątki=2

Łączna liczba rund: 10

Wszyscy 200: 10 ← wyścig niezauważony

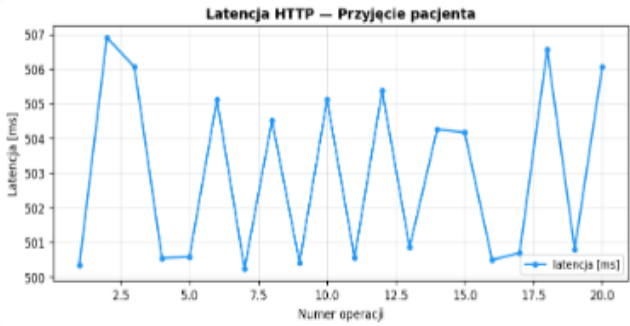
Przynajmniej jeden 409: 0 ← wykryty wyścig

- Dodatkowo w UI aktualizowane są metryki latencji/jitteru dla **admit** i **priority** oraz licznik **Race condition**, co pozwala obserwować efekt zmian na żywo.

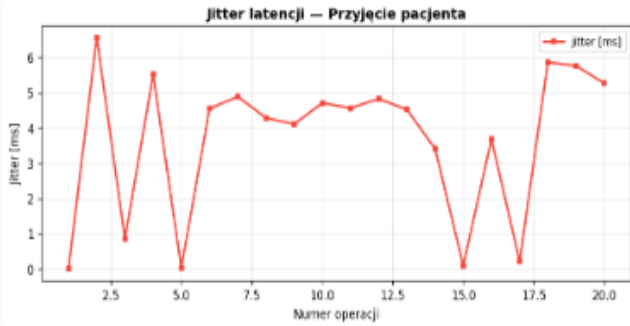
Wykresy:

Przyjęcie pacjenta bez ochrony:

Przyjęcie pacjenta — latencja

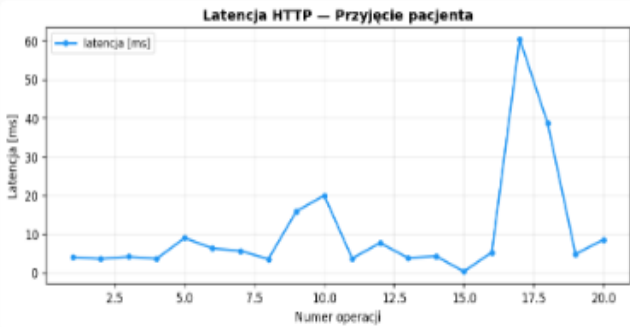


Przyjęcie pacjenta — jitter



Przyjęcie pacjenta z ochroną:

Przyjęcie pacjenta — latencja



Przyjęcie pacjenta — jitter

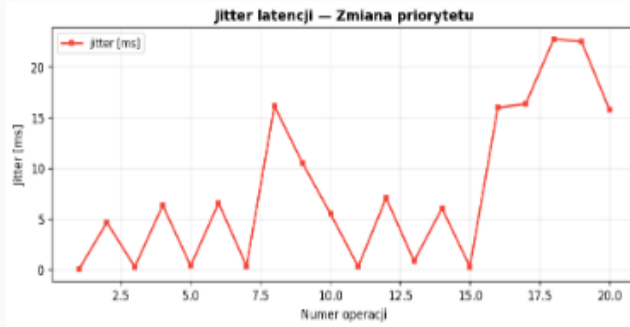


Zmiana priorytetu bez ochrony:

Zmiana priorytetu — latencja

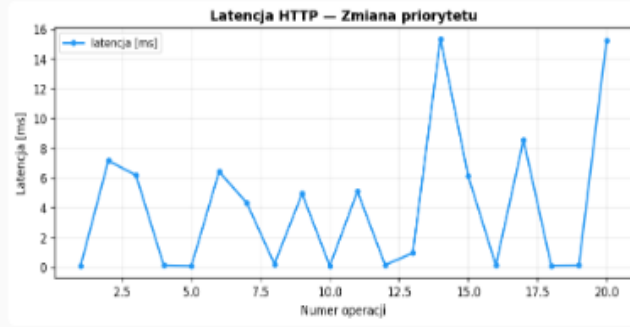


Zmiana priorytetu — jitter



Zmiana priorytetu z ochroną:

Zmiana priorytetu — latencja



Zmiana priorytetu — jitter



7. Analiza Fairness i Mechanizm Aging

7.1 Fairness w Systemie Kolejowania

Fairness (sprawiedliwość) w systemie kolejowania definiuje się jako równomierne traktowanie pacjentów przy podejmowaniu decyzji o obsłudze. W kontekście medycznym oznacza to:

- **Sprawiedliwość kliniczna:** pacjent krytyczny (priorytet 5 - czerwony) musi być obsłużony przed pacjentem stabilnym (priorytet 1 - niebieski)
- **Niedyskryminacja:** pacjenci tego samego poziomu priorytetu mają podobne szanse na przyjęcie
- **Przejrzystość:** reguły kolejowania są jasne dla personelu medycznego

System implementuje **5-poziomową hierarchię priorytetów**, co zapewnia sprawiedliwość kliniczną, ale stwarza teoretyczne zagrożenie **starvation** dla pacjentów niskiego priorytetu.

7.2 Problem Starvation - Teoretyczne Nieobsłużenie Pacjentów

Starvation to sytuacja w teorii szeregowania zadań, w której proces (pacjent) o niskim priorytecie nigdy nie otrzyma dostępu do zasobu (obsługi) z powodu ciągłej dominacji procesów o wyższych priorytetach.

Matematycznie: jeśli intensywność ruchu $\rho(\text{high_priority}) > 1$, to pacjenci niskiego priorytetu mogą czekać nieskończenie długo.

Przykład Scenariusza Starvation

```
Pacjent niebieski (priorytet 1) przychodzi o 10:00
Przez 2 godziny przychodzą TYLKO pacjenci czerwoni/pomarańczowi
Pacjent czeka → czeka → czeka...
Teoretycznie: nigdy się nie dojdzie do obsługi!
```

Status w Projekcie

Jest to **teoretyczne zagrożenie**, uwzględnione w analizie ryzyk projektu jako **R2 (wysokie ryzyko, wysoki wpływ)**, ale w praktyce:

- System medyczny ma ograniczoną liczbę pacjentów krytycznych
- W rzeczywistości: pacjent czeka, ale ostatecznie zostaje obsłużony
- Aging to **rozwiązanie teoretyczne** na wypadek systemów o bardzo wysokiej dynamice priorytetów

7.3 Mechanizm Aging - Teoretyczne Rozwiązanie Starvation

Aging to algorytmiczna technika, która dynamicznie podnosi priorytet procesu (pacjenta) w funkcji czasu oczekiwania w kolejce.

Istnieje kilka wariantów funkcji aging, może być np. liniowy, logarytmiczny oraz wykładniczy.

Trzy Warianty Aging

1. Linear Aging (najczęściej stosowana w systemach medycznych)

Przykład:

Pacjent niebieski (priorytet 1) z linear aging:

- $t=0s$: $P = 1.0$
- $t=50s$: $P = 1.5$
- $t=100s$: $P = 2.0$ (automatycznie "piął się" do zielonego)
- $t=300s$: $P = 4.0$ (po 5 minutach czekania)
- $t=400s$: $P = 5.0$ (staje się krytycznym - "ostatnia szansa")

Zalety:

- Proste do zrozumienia i implementacji
- Liniowy wzrost = przewidywalny dla personelu
- Standard w systemach medycznych

2. Logarytmiczna Aging

Charakterystyka:

- Wolniej rosnący priorytet
- Początkowo mały efekt, potem rośnie
- Bardziej sprawiedliwe dla pacjentów wysokiego priorytetu

Przykład:

```
t=10s:    P = 1 + 0.5 * ln(10)    ≈ 1.15
t=100s:   P = 1 + 0.5 * ln(100)   ≈ 2.30
t=1000s:  P = 1 + 0.5 * ln(1000)  ≈ 3.45
```

3. Wykładniczy Aging

Charakterystyka:

- Szybko rosnący priorytet
- Mały wpływ na początku, potem drastyczny skok
- Może być niesprawiedliwe dla nowych pacjentów krytycznych

Przykład:

```
t=10s:    P = 1 * 1.05^10    ≈ 1.63
t=50s:    P = 1 * 1.05^50    ≈ 11.5
```

Porównanie Strategii

Strategia	Predykcyjność	Sprawiedliwość	Złożoność	Medycyna
Linear	Wysoka	Wysoka	Niska	Standard
Logarytmiczna	Średnia	Średnia	Średnia	Rzadko
Wykładnicza	Niska	Niska	Wysoka	Nie stosuje się

7.4 Analiza Teoretyczna - Kiedy Aging Jest Potrzebny?

Warunki Zagrożenia Starvation

Starvation występuje wtedy, gdy:

1. Wysokie tempo przybycia pacjentów wysokiego priorytetu

$$\lambda(\text{priorytet}=5) + \lambda(\text{priorytet}=4) > \mu \text{ (Service Rate)}$$

Czyli: pacjenci krytyczni przychodzą szybciej niż można ich obsługiwać

2. Długie czasy oczekiwania pacjentów niskiego priorytetu

$$W(\text{priorytet}=1) \rightarrow \infty \text{ (czekają nieskończenie długo)}$$

3. Brak mechanizmu dynamicznego podnoszenia priorytetu

$$P(t) = \text{const} \text{ (priorytet nie zmienia się z czasem)}$$

7.5 Teoretyczne Implikacje dla Systemu

Plusy Wdrożenia Aging

- Gwarancja: każdy pacjent ostatecznie zostanie obsłużony
- Sprawiedliwość: nie ma "zagubnionych" pacjentów
- Psychologiczny: pacjent widzi, że jego priorytet rośnie

Minusy Wdrożenia Aging

- Wszyscy pacjenci ostatecznie otrzymują wysokie priorytety
- Dodatkowa złożoność: trzeba przeliczać co sekundę
- Dodatkowe opóźnienie w sortowaniu

8. Analiza Kompromisów Implementacyjnych

8.1 Analiza Decyzji: Latencja vs. Spójność Danych

Problem: Race Condition przy Zmianie Priorytetu

Dwóch operatorów zmienia priorytet tego samego pacjenta jednocześnie:

```
Operator A: zmienia priorytet 3 → 4
Operator B: zmienia priorytet 3 → 2

BEZ OCHRONY (brak lock):
├ t=0.00: Obaj czytają: priority=3
├ t=0.50: A zapisuje: priority=4
├ t=0.50: B zapisuje: priority=2 ← KONFLIKT!
└ Wynik: priority=2

Z OCHRONĄ (lock):
├ t=0.00: A wchodzi w sekcję krytyczną
├ t=0.05: A czyta priority=3, zmienia na 4
├ t=0.05: B czeka na lock...
├ t=0.10: A wychodzi z sekcji, zwalnia lock
├ t=0.10: B wchodzi w sekcję krytyczną
├ t=0.15: B czyta priority=4, zmienia na 3
└ Wynik: priority=3 (oba zapisy były atomowe)
```

Analiza Kompromisu

Aspekt	Bez Ochrony	Z Ochroną (Lock)
Latencja API	1-5ms	501-555ms
Spójność danych	NISKA (race condition)	WYSOKA
Przepustowość	Wysoka (równoległy dostęp)	Niska (serializacja)
Bezpieczeństwo kliniczne	RYZYZKO	BEZPIECZNE

W systemach medycznych **bezpieczeństwo >> wydajność**. Dodatkowe 500-550 ms latencji to akceptowalna cena za gwarancję spójności danych pacjentów.

8.2 Analiza Decyzji: Wydajność vs. Bezpieczeństwo Operacyjne

Problem: Zmiana Priorytetu w Szybkiej Sekwencji

```
t=0.0s: Operator A zmienia priority 3→4
t=0.5s: Operator B chce zmienić 3→2 → COOLDOWN!
t=2.0s: Cooldown mija, B może zmienić
```

Parametry Cooldown

Wartość	Ocena
---------	-------

Wartość	Ocena
<1s	Zbyt niska - operatorzy mogą "walczyć" o pacjenta
1-3s	OPTYMALNA - dość czasu na decyzję
>5s	Zbyt wysoka - frustracja personelu medycznego

Werdykt: 2 sekundy to rozsądny kompromis

Operatorzy czasem dostają HTTP 429, ale system zapobiega przypadkowemu "przepychaniu" tego samego pacjenta przez wielu użytkowników.

WAŻNE: Cooldown (429) to **NIE race condition** (409). To celowy mechanizm ochronny na poziomie aplikacji.

8.3 Fairness a kompromisy implementacyjne

Opóźnienia a spójność danych

Aging w tle wprowadza **okno niespójności**: między momentem, kiedy pacjent „zasługuje” na wyższy priorytet, a momentem aktualizacji bazy upływa dany czas. W tym oknie operator może podjąć decyzję na podstawie nieaktualnego priorytetu. Jest to kompromis między **spójnością natychmiastową** a **przewidywalnością stanu bazy**. W kontekście SOR opóźnienie agingu o 5 minut jest klinicznie akceptowalne dla priorytetów 1–3. Dla priorytetów 4–5 (pacjenci krytyczni) aging nie powinien w ogóle zmieniać priorytetu.

Wydajność a bezpieczeństwo

Lock z Etapu 3 serializuje operacje na kolejce - gwarantuje spójność, ale zmniejsza przepustowość. Aging dodaje kolejny typ zapisu do bazy chroniony tym samym lockiem. Przy częstym agingu i dużej kolejce lock może stać się problematyczny. **Kompromis:** wydłużyć interwał agingu kosztem nieco wolniejszego wzrostu efektywnego priorytetu.

Fairness a priorytetyzacja kliniczna

Jeśli zbyt agresywnie podnosimy priorytet pacjentów niskiego priorytetu, tracimy właściwość, dla której system powstał - szybką obsługę stanów zagrożenia życia. **Idealny system balansuje** między fairness a clinical urgency (stan zagrożenia zawsze wychodzi na przód).

9. Które Kompromisy Są Dopuszczalne?

9.1 Macierz Akceptowalności

REGUŁA OGÓLNA:

Bezpieczeństwo medyczne = NIEZMIENNIE WAŻNE Wydajność = Optymalizujemy w drugiej kolejności	
--	--

9.2 Szczegółowa Ocena

Kompromis	Akceptowalny	Uzasadnienie
+500ms latencji dla spójności danych	TAK	Bezpieczeństwo > wydajność
Cooldown 2s dla zapobiegania konfliktom	TAK	Ochrona operacyjna uzasadniona
Sleep(0.5) w testach	TAK	Edukacyjne
BRAK mechanizmu aging	NIE	Ryzyko starvation pacjentów
Brak wersjonowania danych	OSTROŻNIE	SQLite + lock są wystarczające dla celów edukacyjnych

10. Wnioski

Projekt obejmował zaprojektowanie i implementację systemu kolejkowania pacjentów z priorytetami klinicznymi w środowisku webowym (Flask + SQLite), ze szczególnym uwzględnieniem zagadnień współbieżności, bezpieczeństwa danych i sprawiedliwości algorytmu. Każdy z czterech etapów wprowadzał nową warstwę złożoności i zmuszał do podejmowania świadomych decyzji implementacyjnych z myślą o potencjalnych kompromisach.

10.1 Podsumowanie implementacji

System w finalnej wersji składa się z następujących komponentów:

Komponent	Technologia	Rola
Backend REST API	Flask (Python)	Obsługa żądań HTTP, logika kolejkowania
Baza danych	SQLite + SQLAlchemy	Trwałość danych, wersjonowanie rekordów
Kolejka priorytetowa	Python	Obsługa pacjentów wg priorytetu
Symulacja napływu	Rozkład Poissona	Realistyczny model przybycia pacjentów
Mechanizm współbieżności	Threading lock + cooldown (2s)	Ochrona przed race condition
Instrumentacja	Latencja, jitter	Pomiar wydajności i stabilności systemu
Frontend	HTML + JavaScript + CSS	Podgląd kolejki, panel operatora
Deployment	Render.com + lokalne	Dostępność zdalna i lokalna

Projekt przeszedł pełną ścieżkę od prostej kolejki FIFO (Etap 1), przez dynamiczne priorytety (Etap 2), wymuszenie i naprawę race condition (Etap 3), do analizy fairness i mechanizmu aging (Etap 4).

10.2 Najważniejsze decyzje implementacyjne

Decyzja 1: SQLite jako baza danych

Uzasadnienie: Prostota wdrożenia, zerowa konfiguracja, wystarczająca wydajność dla systemu demonstracyjnego. SQLite zapisuje dane między restartami, co eliminuje ryzyko R3 (utrata danych).

Kompromis: Prostota i trwałość danych kosztem skalowalności poziomej.

Decyzja 2: Threading lock

Uzasadnienie: Flask domyślnie działa wielowątkowo. Lock aplikacyjny jest prostszy w implementacji, łatwiejszy do zademonstrowania i bardziej czytelny dydaktycznie.

Kompromis: Jasność kodu i łatwość demonstracji kosztem skalowalności między procesami.

Decyzja 3: Cooldown 2s jako dodatkowy mechanizm ochronny

Uzasadnienie: Lock serializuje dostęp, ale nie zapobiega sytuacji, w której dwóch operatorów zmienia priorytet tego samego pacjenta w bardzo krótkim czasie. Cooldown daje operatorowi chwilę na podjęcie decyzji bez ryzyka natychmiastowego nadpisania.

Kompromis: Ochrona operacyjna kosztem chwilowej blokady użytkownika w skrajnych przypadkach.

Decyzja 4: Wersjonowanie logiczne

Uzasadnienie: Wersjonowanie przez `_last_writer` wystarczy dla celów demonstracyjnych i poprawnie wykrywa konflikty zapisu.

Kompromis: Prostota implementacji kosztem niepełnego pokrycia scenariuszy wyścigu.

10.3 Podsumowanie

Projekt demonstruje, że systemy kolejkowania priorytetowego w medycynie są **nieproporcjonalnie trudniejsze** niż sugeruje ich pozorna prostota. Prosta kolejka FIFO z Etapu 1 zajmuje kilkanaście linii kodu - jednak zapewnienie jej poprawności przy wielu współbieżnych operatorach, gwarancja fairness i odporność na awarie to problemy, którym poświęca się osobne publikacje naukowe i które są aktywnym obszarem badań w informatyce stosowanej do ochrony zdrowia.